ASSIGNMENT 2 - TASK 2

Report on Implementing Linear Regression Using a
Multi-Layer Neural Network From Scratch

Student Name	**Details**
Student Name	Sushmita Halder, Jyoti, Rajeev Kumar
Roll Number	2621MA08, 2621MA12, 2621MA01
Group	08
Course	Large Language Models
Submission Date	01/09/2026

1. Introduction

This project implements a feed-forward multi-layer neural network for a regression problem using the California Housing dataset. The task predicts the median house value (MedHouseVal) from two selected features: Median Income (MedInc) and House Age (HouseAge).

The main requirement is to understand the neural-network learning process by implementing the model and optimization procedures from scratch rather than using pre-built model functions from libraries such as sklearn or PyTorch. The supplied notebook uses sklearn only for dataset loading, while the neural-network classes, activations, forward propagation, loss, backpropagation and optimizers are implemented with NumPy.

The baseline architecture is $2 \rightarrow 5 \rightarrow 3 \rightarrow 1$. Basic Gradient Descent, SGD with Momentum, and Adam are compared. Bonus experiments investigate an additional hidden layer and L2 regularization.

2. Objectives

- Use MedInc and HouseAge as the two input features.
- Normalize the input features and split the data into 80% training and 20% testing sets.
- Build the required $2 \rightarrow 5 \rightarrow 3 \rightarrow 1$ neural-network architecture.
- Implement ReLU and linear activations from scratch.
- Implement forward propagation and backpropagation manually.
- Train using Basic Gradient Descent with learning rates 0.01 and 0.001.
- Compare Basic Gradient Descent, Momentum, and Adam at learning rate 0.001.
- Evaluate the final model using test-set MSE and actual-versus-predicted visualization.
- Study the effect of an additional hidden layer.
- Study L2 regularization using $\lambda = 0.001$ and $\lambda = 0.01$.

3. Experimental Setup

3.1 Dataset and Feature Selection

The dataset contains 20,640 observations. The notebook selects two features, MedInc and HouseAge, and uses MedHouseVal as the regression target.

Item	Recorded value
Selected feature matrix	(20640, 2)
Target vector	(20640,)
Training samples	16,512
Testing samples	4,128
Train/test split	80% / 20%

3.2 Normalization

The original submitted notebook records standardized feature values. The first five rows printed by the notebook are reproduced exactly below.

Recorded normalized-feature output

First 5 normalized training rows:

```
[[-1.15453971 -0.29052552]
 [-0.7062249   0.10661646]
 [-0.20585896  1.85404119]
 [ 0.98467237 -0.9259527 ]
 [-0.07670674  0.42433005]]
```

For the corrected from-scratch notebook, normalization is performed manually using the training-set mean and standard deviation. The same training statistics are then applied to the test data.

3.3 Network Architecture

Layer	Neurons	Activation
Input	2	None
Hidden Layer 1	5	ReLU
Hidden Layer 2	3	ReLU
Output	1	Linear

Architecture: 2 → 5 → 3 → 1

4. Methods and Mathematical Formulation

4.1 Dense Layer

Every fully connected layer calculates the affine transformation:

$Z = XW + b$

The weights are initialized with random values, with He-style scaling used for ReLU layers. Biases are initialized to zero.

4.2 ReLU Activation

$\mathrm{ReLU}(z) = \max(0,z)$

4.3 Linear Output

The final regression neuron uses a linear activation, $f(z)=z$, so its derivative is 1.

4.4 Mean Squared Error

$\mathrm{MSE} = (1/n)\,\Sigma(y - \hat{y})^2$

The gradient used for backpropagation is $\partial \mathrm{MSE}/\partial \hat{y} = (2/n)(\hat{y} - y)$.

4.5 Backpropagation

For a dense layer the parameter gradients are calculated as:

$dW = X^T dZ, \quad db = \Sigma dZ, \quad dX = dZW^T$

5. Optimization Methods

A. Basic Gradient Descent

$w(t+1) = w(t) - \eta \nabla L(w(t))$

The experiment uses learning rates 0.01 and 0.001.

B. SGD with Momentum

$v(t) = \beta v(t-1) + (1-\beta)g(t)$

The implementation uses $\beta=0.9$.

C. Adam

$m(t) = \beta_1 m(t-1) + (1-\beta_1)g(t)$

$v(t) = \beta_2 v(t-1) + (1-\beta^2)g(t)^2$

$\hat{m}(t) = m(t)/(1-\beta_1\text{^}t), \quad \hat{v}(t) = v(t)/(1-\beta 2\text{^}t)$

$w(t+1) = w(t) - \eta\, \hat{m}(t)/(\sqrt{\hat{v}}(t)+\varepsilon)$

The recorded experiment uses $\beta 1=0.9$, $\beta 2=0.999$ and $\varepsilon=10^{-8}$.

5. Basic Gradient Descent — Learning-Rate Experiment

The notebook trains the same network for 1,000 epochs using learning rates 0.01 and 0.001. The complete recorded console output is included below.

Complete Basic Gradient Descent console output

Basic Gradient Descent | Learning Rate = 0.01

Epoch 1/1000 | Training MSE = 3.014886

Epoch 100/1000 | Training MSE = 0.689151

Epoch 200/1000 | Training MSE = 0.657615

Epoch 300/1000 | Training MSE = 0.648658

Epoch 400/1000 | Training MSE = 0.643154

Epoch 500/1000 | Training MSE = 0.639882

Epoch 600/1000 | Training MSE = 0.637845

Epoch 700/1000 | Training MSE = 0.636607

Epoch 800/1000 | Training MSE = 0.635776

Epoch 900/1000 | Training MSE = 0.635214

Epoch 1000/1000 | Training MSE = 0.634849

Basic Gradient Descent | Learning Rate = 0.001

Epoch 1/1000 | Training MSE = 3.014886

Epoch 100/1000 | Training MSE = 1.738311

Epoch 200/1000 | Training MSE = 1.293374

Epoch 300/1000 | Training MSE = 1.058341

Epoch 400/1000 | Training MSE = 0.917815

Epoch 500/1000 | Training MSE = 0.830625

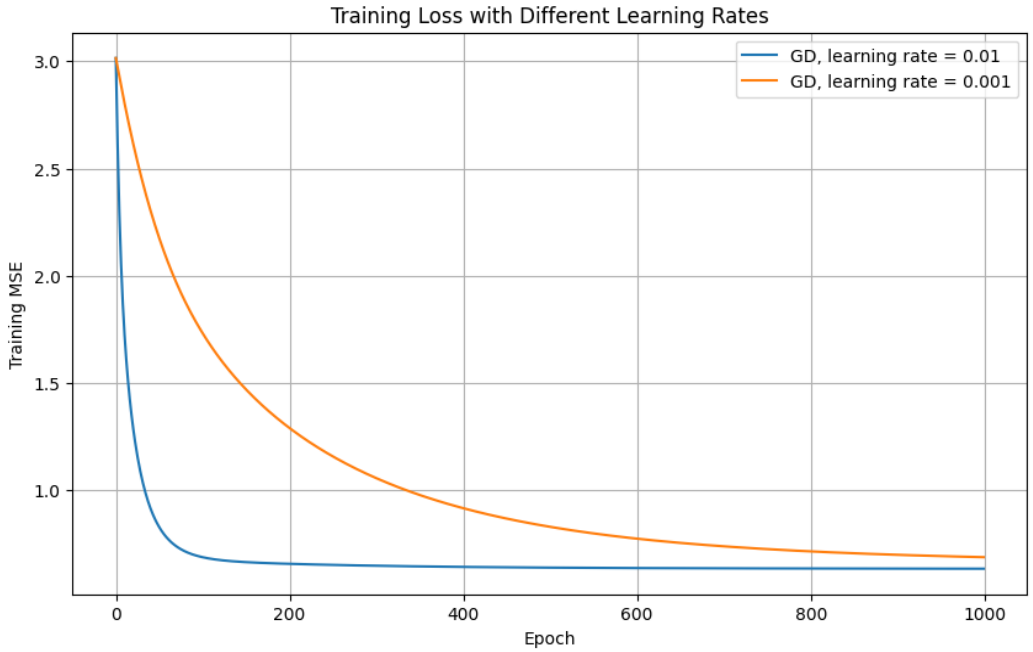
Epoch 600/1000 | Training MSE = 0.775172

Epoch 700/1000 | Training MSE = 0.739364

Epoch 800/1000 | Training MSE = 0.715635

Epoch 900/1000 | Training MSE = 0.699619

Epoch 1000/1000 | Training MSE = 0.688571



The recorded result shows that 0.01 produces much faster loss reduction within the 1,000 epochs.

6. Optimizer Comparison

The notebook compares Basic Gradient Descent, Momentum, and Adam at a fixed learning rate of 0.001. Each optimizer is trained for 1,000 epochs.

Optimizer: GD | Learning Rate = 0.001

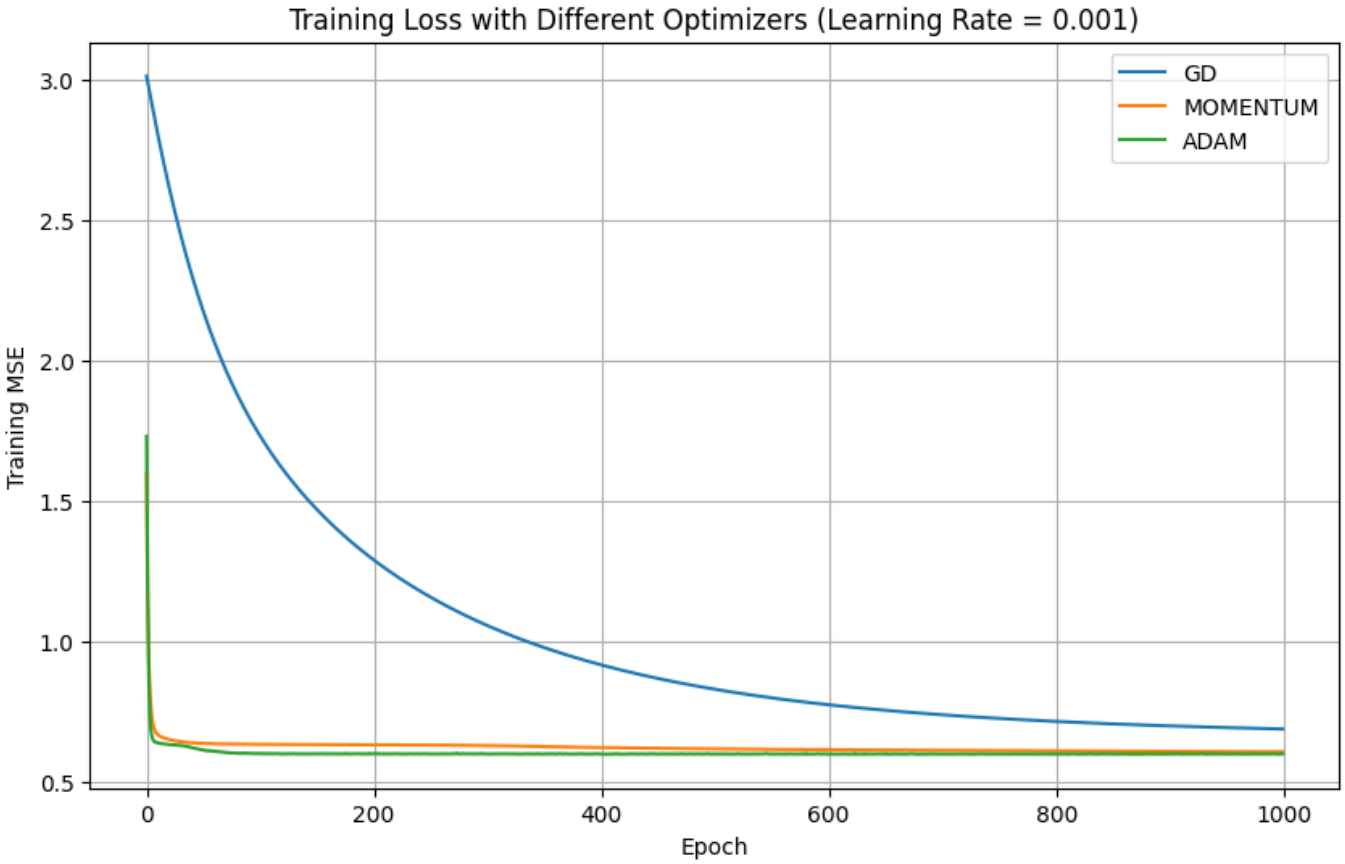
Epoch 1/1000 | Training MSE = 3.014886
Epoch 100/1000 | Training MSE = 1.738311
Epoch 200/1000 | Training MSE = 1.293374
Epoch 300/1000 | Training MSE = 1.058341
Epoch 400/1000 | Training MSE = 0.917815
Epoch 500/1000 | Training MSE = 0.830625
Epoch 600/1000 | Training MSE = 0.775172
Epoch 700/1000 | Training MSE = 0.739364
Epoch 800/1000 | Training MSE = 0.715635
Epoch 900/1000 | Training MSE = 0.699619
Epoch 1000/1000 | Training MSE = 0.688571

Optimizer: MOMENTUM | Learning Rate = 0.001

Epoch 1/1000 | Training MSE = 1.598282
Epoch 100/1000 | Training MSE = 0.634119
Epoch 200/1000 | Training MSE = 0.632114
Epoch 300/1000 | Training MSE = 0.629651
Epoch 400/1000 | Training MSE = 0.622387
Epoch 500/1000 | Training MSE = 0.617865
Epoch 600/1000 | Training MSE = 0.614539
Epoch 700/1000 | Training MSE = 0.612039
Epoch 800/1000 | Training MSE = 0.610169
Epoch 900/1000 | Training MSE = 0.608623
Epoch 1000/1000 | Training MSE = 0.607227

Optimizer: ADAM | Learning Rate = 0.001

Epoch 1/1000 | Training MSE = 1.731484
Epoch 100/1000 | Training MSE = 0.601218
Epoch 200/1000 | Training MSE = 0.599894
Epoch 300/1000 | Training MSE = 0.600099
Epoch 400/1000 | Training MSE = 0.599376
Epoch 500/1000 | Training MSE = 0.599693
Epoch 600/1000 | Training MSE = 0.599285
Epoch 700/1000 | Training MSE = 0.599978
Epoch 800/1000 | Training MSE = 0.599403
Epoch 900/1000 | Training MSE = 0.600008
Epoch 1000/1000 | Training MSE = 0.599846



Adam reaches the lowest recorded training loss at epoch 1,000. Momentum slightly improves on plain GD.

7. Evaluation of the Best Model

The notebook retrains the selected Adam configuration and evaluates it on the held-out test set.

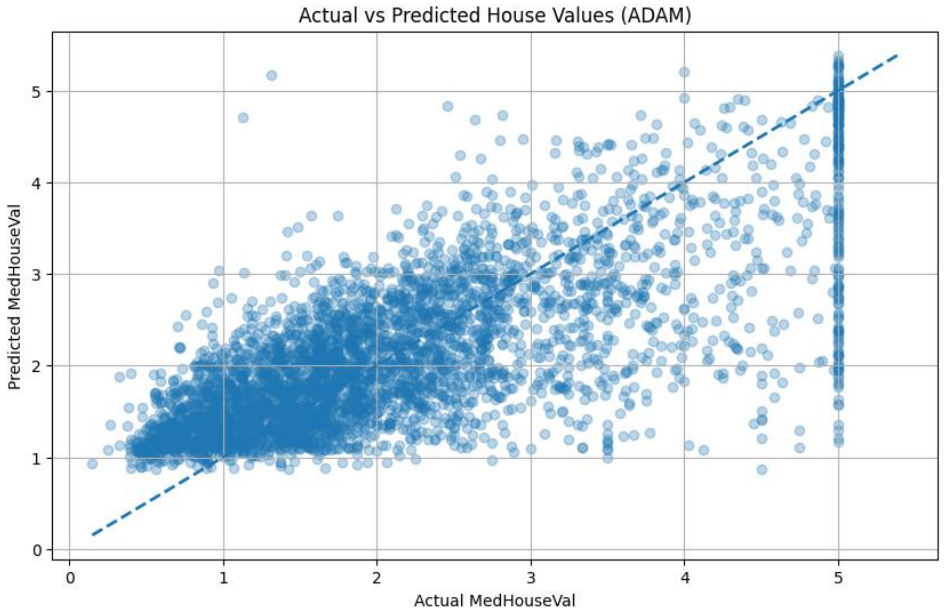
GD | Train MSE = 0.688480 | Test MSE = 0.710231

MOMENTUM | Train MSE = 0.607227 | Test MSE = 0.630170

ADAM | Train MSE = 0.599846 | Test MSE = 0.621427

Best optimizer: **ADAM**

Best test MSE: 0.6214272139540666

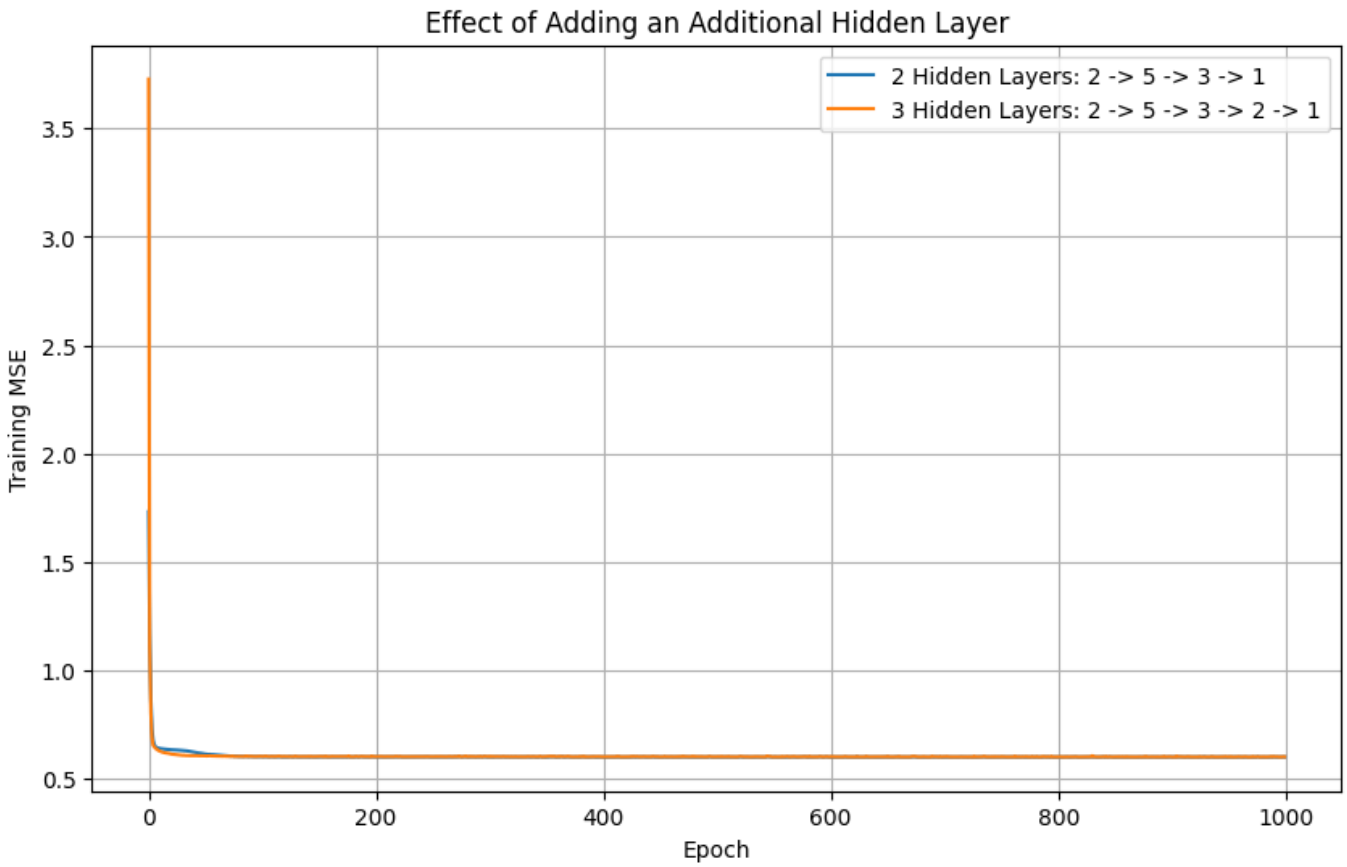


8. Bonus Experiment — Additional Hidden Layer

The required baseline network is 2 → 5 → 3 → 1. The bonus experiment adds a third hidden layer with two neurons, giving 2 → 5 → 3 → 2 → 1.

Complete 3-hidden-layer console output

```
BONUS: 3-HIDDEN-LAYER NETWORK

Epoch    1/1000 | Training MSE = 3.728264

Epoch  100/1000 | Training MSE = 0.602087

Epoch  200/1000 | Training MSE = 0.600996

Epoch  300/1000 | Training MSE = 0.601165

Epoch  400/1000 | Training MSE = 0.600454

Epoch  500/1000 | Training MSE = 0.600565

Epoch  600/1000 | Training MSE = 0.600344

Epoch  700/1000 | Training MSE = 0.600915

Epoch  800/1000 | Training MSE = 0.600494

Epoch  900/1000 | Training MSE = 0.600086

Epoch 1000/1000 | Training MSE = 0.600483


Test MSE - 2 hidden layers: 0.6214272139540666

Test MSE - 3 hidden layers: 0.6219131049623366
```



Recorded output figure: Additional Hidden Layer Comparison

Architecture	Recorded test MSE
2 → 5 → 3 → 1	0.6214272139540666
2 → 5 → 3 → 2 → 1	0.6219131049623366

The additional hidden layer did not improve the recorded test performance.

9. Bonus Experiment — L2 Regularization

L2 regularization adds a penalty for large weights.

$L = MSE + \lambda \Sigma W^2$

The notebook evaluates λ=0.001 and λ=0.01 using Adam with learning rate 0.001.

Complete L2 regularization console output

L2 Regularization | lambda = 0.001

Epoch 1/1000 | Regularized Loss = 1.744452 | MSE = 1.731243

Epoch 100/1000 | Regularized Loss = 0.610367 | MSE = 0.601974

Epoch 200/1000 | Regularized Loss = 0.608565 | MSE = 0.601220

Epoch 300/1000 | Regularized Loss = 0.607789 | MSE = 0.600602

Epoch 400/1000 | Regularized Loss = 0.607036 | MSE = 0.599944

Epoch 500/1000 | Regularized Loss = 0.607595 | MSE = 0.600617

Epoch 600/1000 | Regularized Loss = 0.606846 | MSE = 0.599930

Epoch 700/1000 | Regularized Loss = 0.607537 | MSE = 0.600693

Epoch 800/1000 | Regularized Loss = 0.606793 | MSE = 0.599964

Epoch 900/1000 | Regularized Loss = 0.607422 | MSE = 0.600596

Epoch 1000/1000 | Regularized Loss = 0.607444 | MSE = 0.600589

Test MSE with lambda=0.001: 0.622505

L2 Regularization | lambda = 0.01

Epoch 1/1000 | Regularized Loss = 1.859017 | MSE = 1.728929

Epoch 100/1000 | Regularized Loss = 0.658456 | MSE = 0.618116

Epoch 200/1000 | Regularized Loss = 0.655351 | MSE = 0.617037

Epoch 300/1000 | Regularized Loss = 0.650953 | MSE = 0.613651

Epoch 400/1000 | Regularized Loss = 0.646244 | MSE = 0.612069

Epoch 500/1000 | Regularized Loss = 0.644469 | MSE = 0.611577

Epoch 600/1000 | Regularized Loss = 0.643636 | MSE = 0.610939

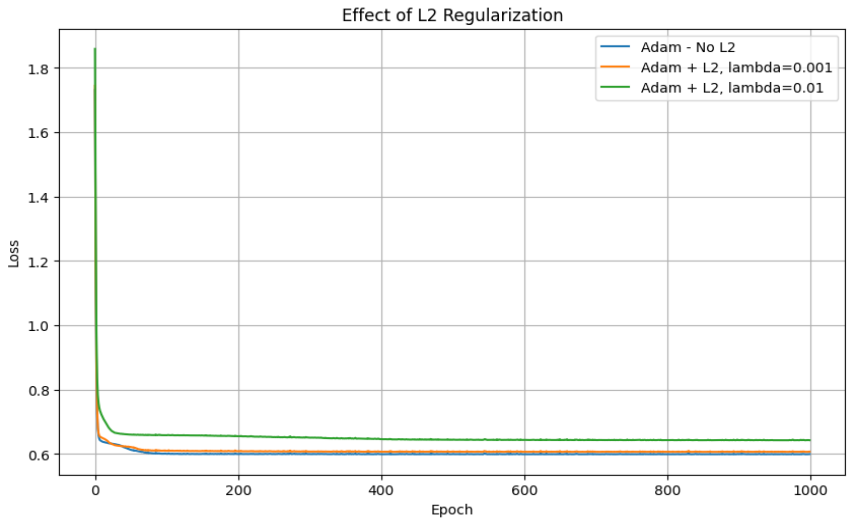
Epoch 700/1000 | Regularized Loss = 0.644204 | MSE = 0.611817

Epoch 800/1000 | Regularized Loss = 0.643223 | MSE = 0.610825

Epoch 900/1000 | Regularized Loss = 0.643175 | MSE = 0.610964

Epoch 1000/1000 | Regularized Loss = 0.642957 | MSE = 0.610551

Test MSE with lambda=0.01: 0.632885



Recorded output figure: Effect of L2 Regularization

Configuration	Recorded test MSE
Adam, no regularization	0.6214272139540666
L2, λ=0.001	0.622505
L2, λ=0.01	0.632885

Neither tested regularization value improves the recorded test MSE compared with the baseline Adam model.

10. Summary and Analysis

10.1 Impact of Learning Rate

A learning rate of 0.01 reduces the training loss than 0.001 in the recorded 1,000-epoch run. The final recorded training losses are 0.6348 and 0.6885 respectively.

10.2 Impact of Optimizers

At learning rate 0.001, Adam achieves the lowest recorded training loss at epoch 1,000 (0.5998). Momentum achieves 0.6072, while GD achieves 0.6348.

10.3 Impact of Additional Hidden Layer

The three-hidden-layer model produces a test MSE of 0.6219 compared with 0.6214 for the baseline two-hidden-layer model.

10.4 Impact of L2 Regularization

The baseline test MSE is 0.622505. L2 with λ=0.001 gives 0.622505 and λ=0.01 gives 0.632885.

11. Key Findings

- The California Housing dataset contains 20,640 observations; the experiment uses two input features.
- The required network architecture is 2 → 5 → 3 → 1.
- Basic GD with learning rate 0.01 gives faster loss reduction than 0.001 in the recorded 1,000-epoch experiment.
- Adam gives the lowest recorded training MSE among GD, Momentum and Adam at LR=0.001.
- The recorded best-model test MSE is 0.6214272139540666.
- Adding a third hidden layer increases the recorded test MSE to 0.6219131049623366.
- L2 regularization with λ=0.001 and λ=0.01 gives test MSE values of 0.622505 and 0.632885 respectively.
- The baseline Adam configuration therefore has the best recorded test performance among the tested configurations.

12. Conclusion

This project demonstrates the complete process of implementing and training a small multi-layer neural network for regression. The essential neural-network operations are implemented from scratch using NumPy.

The experiments show that optimizer and learning-rate choices have a substantial effect on convergence. In the recorded results, Adam provides the best training convergence at learning rate 0.001, and the corresponding baseline network achieves a test MSE of 0.6214272139540666. Neither increasing the network depth nor adding the tested L2 regularization strengths improves that recorded test result.